



Abschlussprojekt Angewantes Machinelles Lernen

Krystian Gluc - 12230903
Cristian Grigore - 12312344
Dominik Berghammer - 12316294
Adrian Linder - 12243106

4. Januar 2026, Vienna

Inhaltsverzeichnis

1 Thema	2
2 Technologien	2
3 Das Ausfüllen von Lückentexten	3
3.1 Einfaches ausfüllen mithilfe von BERT	3
3.2 Verschiedene Modelle für Verschiedene Themengebiete	4
4 Finetune Beispiel mit BERT	5
4.1 Dataset	5
4.2 Training	6
5 Korrektur von Lückentexten	8
5.1 Datenset für Korrektur	8
5.2 Korrektur mit BERT	9

1 Thema

Nach unserem ersten Versuch allgemeiner angelegte Methoden zur Textgenerierung zu bewerten und von Scratch zu trainieren, haben wir uns auf eine spezifischere Aufgabe fokussiert: Masked-Language-Models und Lückentexte. Zu diesen Thema wurden dann 3 Verschiedene Aufgaben spezifiziert

1. Das Ausfüllen von Lückentexten
2. Das Finetuenen von MLMs
3. Das Korrigieren von Lückentexten

2 Technologien

Nach einer Recherche zu diesem Thema, stößt man schnell auf den Begriff Masked-Language-Models. Eines der bekanntesten Models dazu ist BERT (Bidirectional Encoder Representations from Transformers). Es gibt zahlreiche andere Modelle, die für Lückentexte verwendet werden können, wie zum Beispiel BART, RoBERTa, ALBERT, ELECTRA, MPNet wobei hinter vielen von diesen Modellen die BERT Architektur steckt.

Gegeben einen maskierten Text, geben solchen Modelle das Wort zurück, dass sich am wahrscheinlichsten hinter der Maske versteckt. Bei dem Satz "[MASK] is the capital of France." würde so ein Modell zum Beispiel das Wort "Paris" vorhersagen.

3 Das Ausfüllen von Lückentexten

3.1 Einfaches ausfüllen mithilfe von BERT

Eine der einfachsten Möglichkeiten, einen Lückentext automatisch auszufüllen, ist die Verwendung von BERT über die *transformers*-Bibliothek.

```
1 from transformers import pipeline
2
3 pipeline_bert = pipeline(
4     task="fill-mask",
5     model="bert-base-uncased",
6     device=0 # benutzt GPU statt CPU
7 )
8 pipeline_bert(
9     "Plants create [MASK] through a process [MASK] as photosynthesis.",
10    top_k=2 # gibt die Top 2 Resultate zurück
11 )
12 )
```

Nachdem man diesen Code ausführt erhält man folgendes Resultat.

```
1 [
2     [{'score': 0.2662782669067383,
3      'token': 2943,
4      'token_str': 'energy',
5      'sequence': '[CLS] plants create energy through a process [MASK] as
6      photosynthesis. [SEP]'},
7     {'score': 0.10305079817771912,
8      'token': 4870,
9      'token_str': 'flowers',
10     'sequence': '[CLS] plants create flowers through a process [MASK] as
11     photosynthesis. [SEP]'}],
12 [
13     [{'score': 0.9980964064598083,
14      'token': 2124,
15      'token_str': 'known',
16      'sequence': '[CLS] plants create [MASK] through a process known as
17      photosynthesis. [SEP]'},
18     {'score': 0.001142986468039453,
19      'token': 2107,
20      'token_str': 'such',
21      'sequence': '[CLS] plants create [MASK] through a process such as
22      photosynthesis. [SEP]'}],
23 ]
```

Da der Lückentext zwei Lücken enthält, bekommt man eine Liste mit zwei Listen zurück. Diese sind jeweils die Resultate zur ersten und zur zweiten Lücke.

Anhand des Scores sieht man klar, dass sich das Modell bei der zweiten Lücke deutlich sicherer fühlt als bei der ersten Lücke.

Das Modell schlägt uns also den folgenden ausgefüllten Lückentext vor:

*Plants create **energy** through a process **known** as photosynthesis.*

3.2 Verschiedene Modelle für Verschiedene Themengebiete

4 Finetune Beispiel mit BERT

Im folgenden Kapitel werden wir uns nun mit einem Anwendungsbeispiel beschäftigen welches ein BERT Modell nutzt um Informationen für Nutzer bereitzustellen.

Wir wollen nun BERT nutzen um einem Benutzer die Möglichkeit zu geben durch Natürliche Fragen Informationen über eine Fiktive Firma zu erhalten. Daher betrachten wir nun einmal unser Trainingsset

4.1 Dataset

Wir betrachten die Fiktive Firma *Example Tech* mit Standorten in Berlin, München und Hamburg. Wir haben nun eine Liste von Mitarbeitern welche in bestimmten Positionen an bestimmten Orten arbeiten

```
1  {
2      "name": "Alice",
3      "role": "CEO",
4      "location": "Berlin",
5      "email": "ceo@exampletech.com"
6  },
7  {
8      "name": "Carol",
9      "role": "CFO",
10     "location": "Munich",
11     "email": "cfo@exampletech.com"
12 },
13 {
14     "name": "Lily",
15     "role": "Accountant",
16     "location": "Hamburg",
17     "email": "accounting.hamburg@exampletech.com"
18 },
19 ....
```

Insgesamt gibt es 24 Personen mit verschiedenen Jobs, Namen und Arbeitsorten. Weiters bietet das Unternehmen noch verschiedene Produkte an

```
1  {
2      "name": "AlphaPhone",
3      "category": "Electronics",
4      "price": 699
5  },
6  {
7      "name": "OrionERP",
8      "category": "Software",
9      "price": 99
10 },
11 {
12     "name": "LightPanel",
13     "category": "Furniture",
14     "price": 149
15 },
16 {
17     "name": "TechSupport+",
18 }
```

```

19         "category": "Service",
20         "price": 39
21     },
22     ....

```

Es gibt gesamt 19 Produkte.

Nun können wir uns Fragen wie wir dieses Datenset in eine Form überführen mit welcher wir BERT trainieren können. Die Idee war nun dies mittels Natürlicher Fragen über das Datenset zu machen. Wir trainieren also auf Text wie

Alice is the CEO working in the Berlin Office

oder

Alice's email address is ceo@exampletech.com

und auch Fragen wie

Where is Alice? In the Berlin Office.

Selbes machen wir für die Produkte. Würden wir nun diese Fragen einfach dem Tokenizer geben hätten wir jedoch ein Problem. Denn z.B. CEO ist *Ein* Token jedoch z.B. Sales Manager *zwei* Token und eine EMail adresse wird sogar in bis zu *fünf* Token aufgespaltet. Daher unsere Lösung, wir fügen einfach einige neue Token hinzu. Dies lässt sich mittels Transformerers sehr einfach machen.

```

1 emails = [
2     e["email"] for e in json.load(open("dataset.json"))["employees"]
3 ]
4
5
6 jobs = [
7     e["role"] for e in json.load(open("dataset.json"))["employees"]
8 ]
9
10
11 products = [
12     e["name"] for e in json.load(open("dataset.json"))["products"]
13 ]
14
15
16 prices = [
17     str(e["price"]) for e in json.load(open("dataset.json"))["products"]
18 ]
19
20 companyName = [json.load(open("dataset.json"))["company"]["name"]]
21 ]
22
23 tokenizer.add_tokens(emails)
24 tokenizer.add_tokens(jobs)
25 tokenizer.add_tokens(products)
26 tokenizer.add_tokens(prices)
27 tokenizer.add_tokens(companyName)
28
29 # WICHTIG Model auf neues Vokabular anpassen
30 model.resize_token_embeddings(len(tokenizer))

```

4.2 Training

Das Training wird lokal mittels PyTorch und Transformerers durchgeführt. Der genutzte PC hat die folgenden Spezifikationen

CPU	AMD Ryzen 7 7700X
GPU	NVIDIA GeForce RTX 3060 12GB
RAM	32 GB - DDR5 4800 MT/s
Betriebssystem	Arch Linux - 6.18.2
CUDA	12.6

Die Transformerers Bibliothek hat glücklicherweise bereits eine Implementation für BERT somit können wir diese einfach nutzen. Zunächst importieren wir den Tokenizer und das Model sowie den Trainer.

```
1 import torch
2 from transformers import (
3     BertTokenizer,
4     BertForMaskedLM,
5     DataCollatorForLanguageModeling,
6     Trainer,
7     TrainingArguments,
8     EarlyStoppingCallback)
```

damit ist schon fast die Hälfte des Coding aufwands geschafft.

Unsere Nächste überlegung ist nun welches Model wir nutzen wollen zum Trainieren. Unsere Wahl fiel auf *bert-base-uncased*. Dieses ist ein Basis BERT Model welche nicht zwischen Groß- und Kleinschreibung unterscheidet. Wie bereits vorher Diskutiert wurden dem Tokenizer die EMail-Adressen, Jobs und Produkte als Token hinzugefügt damit wir uns nicht über die genaue Anzahl an Mask gedanken machen müssen. Um nun das Model und den Tokenizer zu laden können wir einfach Transformerers nutzen

```
1 model = BertForMaskedLM.from_pretrained("bert-base-uncased")
2 tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

Transformerers wird uns nun automatisch das Model herunterladen und lokal speichern. Dies kann beim ersten Start zu einiger Wartezeit führen je nach Internetvertrag.

Wir wollen nun den Trainer vorbereiten. Hirzu nutzen wir die *TrainingArguments*. Wir nutzen hierbei die folgenden Argumente:

Argument	Wert	Beschreibung
num_train_epochs	500	Anzahl der Epochen (wie oft durch das gesamte Trainingsset durchgegangen wird)
per_device_train_batch_size	16	Die Batch Größe
learning_rate	5e-5(10 ⁻⁵)	Lernrate
save_steps	3000	Wie oft er einen zwischenstand Speichert
save_total_limit	2	Wie viele zwischenstände er behält
logging_steps	100	Wie oft der Loss berechnet und ausgegeben wird
lr_scheduler_type	linear	Wie die Lernrate verändert wird
warmup_ratio	0.1	

Nun können wir also mit diesen Argumenten einen Trainer erstellen.

```
1 training_args = TrainingArguments(
2     #Arguments
3 )
4
5 trainer = Trainer(
6     model=model,
7     args=training_args,
8     train_dataset=dataset,
9     data_collator=data_collator,
10 )
```

Nun sind wir endlich bereit den Trainer zu starten. Dies ist simple und einfach. Anschließend können wir das Model noch speichern damit wir es später verwenden können.

```
1 trainer.train()
2
3 model.save_pretrained("./bert-company")
4 tokenizer.save_pretrained("./bert-company")
```

5 Korrektur von Lückentexten

In diesem Teil wollten wir einem Modell einen ausgefüllten Lückentext geben, damit es ihn korrigiert: Wurde der Text richtig oder falsch ausgefüllt?

5.1 Datenset für Korrektur

Zunächst wurde hier ein öffentliches Datenset "Cloze-test-with-Bert"¹ verwendet. Dieses besteht aus einem Test- und einem Trainingsset, wobei das Testset 1.267 und das Trainingsset 40.398 Sätze mit jeweils zwei Antwortmöglichkeiten beinhaltet. Eine Datei aus dem Datenset sieht z.B. so aus:

```
1 {
2   "qID": "3FBFUUYRMJCQIM5XJOW8CIQYEGA6Z-1",
3
4   "sentence": "Jane received a pet tortoise and an aquarium as a
   birthday gift, but the _ was too small.",
5
6   "option1": "aquarium",
7
8   "option2": "tortoise",
9
10  "answer": "1"
11 }
```

Um das Datenset gut benutzen zu können haben wir es mithilfe Python transformiert:

```
1 input_file = "./test.jsonl"
2 output_file = "./testnew.jsonl" #oder "./train.jsonl" und "./trainnew.jsonl"
3
4 with open(input_file, "r", encoding="utf-8") as fin, \
5     open(output_file, "w", encoding="utf-8") as fout:
6
7     for line in fin:
8         data = json.loads(line)
9
10        sentence = data["sentence"]
11        masked_sentence = sentence.replace("_", "[MASK]") #da BERT [MASK] braucht
12        opt1 = data["option1"]
13        opt2 = data["option2"]
14        correct = data["answer"] # "1" oder "2"
15
16        filled_1 = sentence.replace("_", opt1) # Option 1 einsetzen
17        example_1 = {
18            "qID": data["qID"],
19            "unfilled_sentence": masked_sentence,
20            "filled_sentence": filled_1,
21            "chosen_option": opt1,
22            "is_correct": correct == "1"
```

¹<https://github.com/TheMaxi7/Cloze-test-with-BERT>

```

23     }
24
25     filled_2 = sentence.replace("_", opt2) # Option 2 einsetzen
26     example_2 = {
27         "qID": data["qID"],
28         "unfilled_sentence": masked_sentence,
29         "filled_sentence": filled_2,
30         "chosen_option": opt2,
31         "is_correct": correct == "2"
32     }
33
34     fout.write(json.dumps(example_1, ensure_ascii=False) + "\n")
35     fout.write(json.dumps(example_2, ensure_ascii=False) + "\n")

```

Dieser Code bringt das Datenset in eine Form, die für unsere Anwendung besser ist. Jede Datei wird aufgeteilt in zwei Dateien. Die obere Beispieldatei wird auf zwei Dateien aufgeteilt:

```

1  {
2      "qID": "3FBFUUYRMJJCQIM5XJOW8CIQYEGA6Z-1",
3
4      "unfilled_sentence": "Jane received a pet tortoise and an aquarium
5      as a birthday gift, but the [MASK] was too small.",
6
7      "filled_sentence": "Jane received a pet tortoise and an aquarium as
8      a birthday gift, but the aquarium was too small.",
9
10     "chosen_option": "aquarium",
11
12     "is_correct": true
13 }
14
15 {
16     "qID": "3FBFUUYRMJJCQIM5XJOW8CIQYEGA6Z-1",
17
18     "unfilled_sentence": "Jane received a pet tortoise and an aquarium
19     as a birthday gift, but the [MASK] was too small.",
20
21     "filled_sentence": "Jane received a pet tortoise and an aquarium as
22     a birthday gift, but the tortoise was too small.",
23
24     "chosen_option": "tortoise",
25
26     "is_correct": false
27 }

```

5.2 Korrektur mit BERT

Zuerst versuchen wir den Text mit einem vortrainierten BERT-Modell zu korrigieren. Wir machen dies, indem wir die Funktion `token_probability` definieren, die einen Satz mit `[MASK]`-Token tokenized, die Position des `[MASK]`-Tokens findet, Logits an der Maskenposition auswählt und dann die Logits mittels Softmax in Wahrscheinlichkeiten umwandelt. Das eingesetzte Wort wird von ihr auch tokenized, wobei ein `ValueError` ausgegeben wird, falls das Wort aus mehr als einem Token besteht, um sie später einfach überspringen zu können, da BERT nur einzelne Tokens bewerten kann. Die Funktion gibt am Ende die bedingte Wahrscheinlichkeit aus, dass das Modell den Token vom eingesetzten Wort an der Maskenposition vorhersagt.

Danach wird für jeden Satz in jeder Zeile der jsonl-Datei mithilfe von `token_probability` die bedingte Wahrscheinlichkeit berechnet, außer sie wird übersprungen, da das eingesetzte Wort aus mehreren Tokens besteht. Schließlich geben wir die Anzahl der verwendeten und übersprungenen Sätze an.

Um einen guten Threshold zu finden, ab welcher Wahrscheinlichkeit wir sagen, dass der Satz korrekt ausgefüllt wurde, nehmen wir 300 Werte zwischen dem kleinsten und größten Wahrscheinlichkeit. Für jeden dieser Wahrscheinlichkeitswerte schauen wir uns an welche Wörter eine höhere Wahrscheinlichkeit haben und überprüfen wie viele davon korrekt sind. Der Threshold mit den meisten richtigen Vorhersagen wird als bester Threshold ausgegeben, zusammen mit der Accuracy unter diesem Threshold.

```
1 import json
2 import torch
3 import numpy as np
4 from transformers import BertTokenizer, BertForMaskedLM
5 import torch.nn.functional as F
6
7     model_name = "bert-base-uncased"
8     tokenizer = BertTokenizer.from_pretrained(model_name)
9     model = BertForMaskedLM.from_pretrained(model_name)
10    model.eval()
11
12    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
13    model.to(device)
14
15    print("Running on:", device)
16
17    input_file = "./trainnew.jsonl"
18
19    scores = []    # p(w | context)
20    labels = []    # is_correct
21    skipped = 0
22
23
24    def token_probability(masked_sentence, word):
25        inputs = tokenizer(masked_sentence, return_tensors="pt").to(device)
26
27        mask_index = torch.where(
28            inputs["input_ids"] == tokenizer.mask_token_id
29        )[1]
30
31        with torch.no_grad():
32            outputs = model(**inputs)
33            logits = outputs.logits
34
35            mask_logits = logits[0, mask_index, :]
36            probs = F.softmax(mask_logits, dim=-1)
37
38            token_ids = tokenizer.convert_tokens_to_ids(
39                tokenizer.tokenize(word)
40            )
41
42            if len(token_ids) != 1:
43                raise ValueError
44
45            return probs[0, token_ids[0]].item()
46
47
48    #Scores sammeln
49
50    with open(input_file, "r", encoding="utf-8") as fin:
51        for line in fin:
52            data = json.loads(line)
53
```

```
54     masked_sentence = data["unfilled_sentence"]
55     chosen_option = data["chosen_option"]
56     is_correct_label = data["is_correct"]
57
58     try:
59         p = token_probability(masked_sentence, chosen_option)
60     except ValueError:
61         skipped += 1
62         continue
63
64     scores.append(p)
65     labels.append(is_correct_label)
66
67 scores = np.array(scores)
68 labels = np.array(labels)
69
70 print(f"Verwendete Samples: {len(scores)}")
71 print(Übersprungene Samples: {skipped})
72
73
74 #Optimalen Threshold finden
75
76 best_acc = 0.0
77 best_threshold = 0.0
78
79 for t in np.linspace(scores.min(), scores.max(), 300):
80     preds = scores > t
81     acc = (preds == labels).mean()
82     if acc > best_acc:
83         best_acc = acc
84         best_threshold = t
85
86 print(f"Best Threshold: {best_threshold:.6f}")
87 print(f"Best Accuracy: {best_acc * 100:.2f}%")
```

Der Code gibt auf dem "Trainingsset" folgendes Resultat aus:

```
Running on: cuda
Verwendete Samples: 74644
Übersprungene Samples: 6152
Best Threshold: 0.056660
Best Accuracy: 52.62%
```

Man sieht, dass die Accuracy auch mit dem besten Threshold nur etwas mehr als 50% beträgt, d.h. dass das Modell kaum besser als Raten ist.

Warum passiert das?